Python Notes

Learn Python by Doing

Study notes, quick checks, 20 mini projects
and 20 full projects to build

Based on the official Python 3 documentation

https://docs.python.org/3/

Edition: October 2026 | Examples work on Python 3.10+

Contents

Learn the Language 6

Getting Started with Python 7

1.1 What can you build with Python? 7

1.2 Installing Python 7

1.3 Two ways to run Python 7

1.4 Three rules to remember from day one 7

Numbers, Text and Variables 9

2.1 Numbers 9

2.2 Variables 9

2.3 Text (strings) 10

2.4 Lists (a first look) 11

2.5 Your first real program 11

Control Flow: Decisions and Loops 13

3.1 if, elif and else 13

3.2 for loops and range() 13

3.3 break, continue and loop else 14

3.4 while loops 14

3.5 pass and match 14

Functions 16

4.1 Defining and calling functions 16

4.2 Default values and keyword arguments 16

4.3 *args and **kwargs 17

4.4 Lambdas, annotations and style 17

Data Structures 19

5.1 More on lists 19

5.2 Tuples 20

5.3 Sets 20

5.4 Dictionaries 21

5.5 Looping techniques 21

5.6 Conditions and comparing sequences 21

Modules and Packages 24

6.1 Writing and importing a module . . . 24
6.2 Running a module as a script . . . 24
6.3 Where does Python look for modules? . . . 25
6.4 Packages . . . 25

Input, Output and Files . . . 27

7.1 Formatting output . . . 27
7.2 Reading and writing files . . . 27
7.3 Saving structured data with JSON . . . 28

Errors and Exceptions . . . 30

8.1 try, except, else, finally . . . 30
8.2 Raising your own exceptions . . . 30
8.3 User-defined exceptions . . . 31
8.4 More tools . . . 31

Classes and Objects . . . 32

9.1 Your first class . . . 32
9.2 Inheritance . . . 32
9.3 Private names and dataclasses . . . 33
9.4 Scopes: where is a name visible? . . . 33
9.5 Iterators and generators . . . 33

A Tour of the Standard Library . . . 36

10.1 Files, folders and the system . . . 36
10.2 Command-line programs with argparse . . . 36
10.3 Text patterns, maths and randomness . . . 36
10.4 Dates and times . . . 36
10.5 Collections and itertools . . . 37
10.6 Quality control: testing . . . 37
10.7 More batteries . . . 38

Virtual Environments, pip and Floating-Point . . . 40

11.1 Virtual environments . . . 40
11.2 Floating-point surprises . . . 40
11.3 What to learn next . . . 41

Twenty Projects to Build . . . 42

How to attack any project . . . 42
Project 1: Rock, Paper, Scissors Tournament . . . 43

Project 2: Smart Calculator . . . 43
Project 3: Password Generator and Checker . . . 44
Project 4: Hangman . . . 44
Project 5: Unit Converter . . . 45
Project 6: Contact Book . . . 45
Project 7: Quiz Engine . . . 46
Project 8: Expense Tracker . . . 46
Project 9: Cipher Tool: Caesar and Vigenere . . . 47
Project 10: Text Statistics Command-Line Tool . . . 47
Project 11: Text Adventure Game . . . 48
Project 12: Tic-Tac-Toe vs the Computer . . . 48
Project 13: Bank Account System . . . 49
Project 14: Library Management System . . . 49
Project 15: Student Grade Manager . . . 50
Project 16: Log File Analyser . . . 50
Project 17: File Organiser and Duplicate Finder . . . 51
Project 18: Sudoku Solver . . . 51
Project 19: Markdown to HTML Converter . . . 52
Project 20: Capstone: Inventory Manager with Tests . . . 52

Python Cheat Sheet . . . 53

Answer Key for the Quick Checks . . . 54

Map of the Official Documentation . . . 56

How to use this book

This book turns the official Python 3 documentation (docs.python.org/3), mainly **The Python Tutorial**, into friendly study notes. Every chapter follows the same pattern so you always know what to do next.

Part of the chapter	What to do
Notes and code	Read, then **type the examples yourself** and change them to see what happens.
NOTE / TIP / WATCH OUT boxes	Short extra facts, shortcuts and common mistakes.
MINI PROJECT boxes (yellow)	Small tasks (20 in total). Build them before moving on.
QUICK CHECK boxes (blue)	A few questions. Write answers down, then compare with the answer key at the back.
Part II: 20 Projects	Larger programs that combine everything. Do them in order.

A suggested study plan

Week	Chapters	Goal
1	1 - 3	Run Python, use numbers, text, decisions and loops
2	4 - 5	Write functions and use lists, dicts, sets and tuples
3	6 - 8	Organise code in modules, read/write files, handle errors
4	9 - 11	Classes, the standard library, virtual environments
5 - 8	Part II	Build the 20 projects, at least 2-3 per week

TIP Spend 30 to 60 minutes every day rather than 6 hours once a week. Consistent practice beats long marathons.

NOTE The official documentation is free and always up to date at https://docs.python.org/3/. Each chapter lists the matching official page, so you can go deeper any time. Code examples assume Python 3.10 or newer.

PART I

Learn the Language

Eleven chapters that follow the structure of the official Python Tutorial, with practice tasks after every topic.

CHAPTER 1

Getting Started with Python

Official docs for this chapter: https://docs.python.org/3/tutorial/interpreter.html

Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Its elegant syntax and dynamic typing make it ideal for scripting and rapid application development on almost every platform.

1.1 What can you build with Python?

- **Automation scripts**: rename files, clean data, send reports.
- **Web backends and APIs**, **data analysis**, **machine learning**, **games**.
- **Tools for yourself**: calculators, trackers, small utilities.

1.2 Installing Python

Download Python from https://www.python.org/ and install it. On Windows, tick "Add Python to PATH" in the installer. Then open a terminal and check the version:

```
$ python --version
Python 3.15.0
```

> **NOTE** On some Linux and macOS systems the command is `python3` instead of `python`. Use whichever one works on your computer, everywhere in this book.

1.3 Two ways to run Python

Interactive mode (REPL). Type `python` and you get a `>>>` prompt where each line you type runs immediately. It is perfect for experiments. To leave, type `exit()` or press Ctrl-D (Linux/macOS) or Ctrl-Z then Enter (Windows).

```
>>> print("Hello, World!")
Hello, World!
>>> 2 + 3
5
```

Scripts. Save code in a file ending with `.py` and run it. Create `hello.py`:

```
# hello.py - my first program
print("Hello, World!")
```

```
$ python hello.py
Hello, World!
```

1.4 Three rules to remember from day one

- **Indentation matters.** Python uses indentation (4 spaces) to group code, not braces.
- **Comments** start with # and are ignored by Python. Use them to explain *why*.
- **Read error messages.** Python tells you the file, the line and the kind of error.

> **TIP** Type every example yourself instead of copy-pasting. Your fingers learn the syntax and you will make (and fix) small mistakes, which is exactly how you learn.

MINI PROJECT 1: Hello, You!

Write `greeter.py` that talks to the user.

Your program should:

- Ask for the user's name with `input()`.
- Ask for the year they were born (convert it with `int()`).
- Print a friendly greeting that includes the name and their approximate age.

Starter hint (try on your own first, peek only if stuck):

```
name = input("What is your name? ")
year = int(input("Which year were you born? "))
age = 2026 - year
print(f"Hello, {name}! You are about {age} years old.")
```

QUICK CHECK: answer these in your notebook (answers at the back)

1. Which command shows the installed Python version?
2. What is the difference between interactive mode and running a script?
3. Which symbol starts a comment?
4. What type of value does `input()` always return?
5. How do you leave the interactive interpreter?

CHAPTER 2

Numbers, Text and Variables

Official docs for this chapter: https://docs.python.org/3/tutorial/introduction.html

This chapter covers the basic building blocks: numbers, strings (text) and lists, and how to store them in variables. Try every example in the interpreter.

2.1 Numbers

Python works like a calculator. Integers (`int`) have no size limit; numbers with a decimal point are floats (`float`).

```
>>> 2 + 2
4
>>> 50 - 5 * 6
20
>>> (50 - 5 * 6) / 4
5.0
>>> 17 / 3        # classic division always returns a float
5.666666666666667
>>> 17 // 3       # floor division discards the fraction
5
>>> 17 % 3        # remainder
2
>>> 5 ** 2        # power
25
>>> 1_000_000     # underscores make big numbers readable
1000000
```

Operator	Meaning	Example	Result
`+ - *`	add, subtract, multiply	`6 * 7`	42
`/`	division (float)	`7 / 2`	3.5
`//`	floor division	`7 // 2`	3
`%`	remainder (modulo)	`7 % 2`	1
`**`	power	`2 ** 8`	256

2.2 Variables

A variable is a name that points to a value. Use `=` to assign. Names are case-sensitive, may contain letters, digits and underscores, and cannot start with a digit.

```
>>> width = 20
>>> height = 5 * 9
>>> width * height
900
>>> price = 3.5
>>> round(price * 3, 1)
10.5
```

TIP In interactive mode the special variable `_` holds the last printed result.

2.3 Text (strings)

Strings can use single or double quotes. Use a backslash to escape special characters, a raw string `r"..."` to ignore escapes (handy for Windows paths) and triple quotes for text that spans several lines.

```
>>> 'spam eggs'
'spam eggs'
>>> 'doesn\'t'                  # escape the quote, or use "doesn't"
"doesn't"
>>> print('C:\some\name')       # \n is a newline!
C:\some
ame
>>> print(r'C:\some\name')      # raw string
C:\some\name
>>> print("""\
... Usage: tool [OPTIONS]
...      -h   Show help
... """)
```

Join strings with `+` and repeat them with `*`:

```
>>> 3 * 'un' + 'ium'
'ununununium'
>>> 'Py' 'thon'                  # neighbouring literals join automatically
'Python'
```

Indexing and slicing

Strings are sequences. Indexes start at **0**; negative indexes count from the end. A slice `[start:stop]` includes `start` but excludes `stop`.

```
>>> word = 'Python'
>>> word[0]
'P'
>>> word[-1]
'n'
>>> word[0:2]
'Py'
>>> word[2:]
'thon'
>>> word[-2:]
'on'
>>> len(word)
6
```

WATCH OUT Strings are **immutable**: `word[0] = "J"` raises a `TypeError`. Build a new string instead: `"J" + word[1:]`.

Useful string methods

Method	What it does	Example
`s.upper()`, `s.lower()`	change case	`"hi".upper()` gives `HI`
`s.strip()`	remove spaces at both ends	`" hi ".strip()` gives `hi`
`s.split(sep)`	split into a list	`"a,b,c".split(",")` gives `['a', 'b', 'c']`
`sep.join(list)`	join a list into a string	`"-".join(["a","b"])` gives `a-b`
`s.replace(a, b)`	replace text	`"hello".replace("l","L")` gives `heLLo`
`s.find(x)`	index of first match or -1	`"hello".find("l")` gives `2`
`s.startswith(x)`	True/False test	`"Hello".startswith("He")` gives `True`

f-strings: the modern way to mix text and values

```
>>> name = "Ada"
>>> score = 92.456
>>> f"Hello {name}, your score is {score:.1f}"
'Hello Ada, your score is 92.5'
```

2.4 Lists (a first look)

A list holds several values in order. Lists are **mutable**: you can change them.

```
>>> squares = [1, 4, 9, 16, 25]
>>> squares[0]
1
>>> squares[-3:]
[9, 16, 25]
>>> squares + [36, 49]
[1, 4, 9, 16, 25, 36, 49]
>>> cubes = [1, 8, 27, 65, 125]   # oops, 4**3 is 64, not 65
>>> cubes[3] = 64
>>> cubes.append(216)
>>> len(cubes)
6
>>> letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> letters[2:5] = ['C', 'D', 'E']    # replace a slice
>>> letters
['a', 'b', 'C', 'D', 'E', 'f', 'g']
>>> letters[2:5] = []                 # remove a slice
>>> letters
['a', 'b', 'f', 'g']
```

2.5 Your first real program

The Fibonacci series, where each number is the sum of the previous two, shows multiple assignment and a `while` loop:

```
a, b = 0, 1
while a < 100:
```

```
    print(a, end=' ')
    a, b = b, a + b
# 0 1 1 2 3 5 8 13 21 34 55 89
```

MINI PROJECT 2: Tip Calculator

Build a calculator that splits a restaurant bill.

Your program should:

- Ask for the bill amount, the tip percentage and the number of people.
- Print the tip, the total and what each person pays, all with 2 decimals.

Starter hint (try on your own first, peek only if stuck):

```
bill = float(input("Bill amount: "))
tip_percent = float(input("Tip percent: "))
people = int(input("Split between how many people? "))
tip = bill * tip_percent / 100
total = bill + tip
print(f"Tip: {tip:.2f}")
print(f"Total: {total:.2f}")
print(f"Each person pays: {total / people:.2f}")
```

MINI PROJECT 3: Text Analyser

Ask the user to type a sentence and report facts about it.

Your program should:

- Number of characters (with and without spaces).
- Number of words (hint: `split()`).
- The sentence in UPPER case and with the words reversed.
- Count of vowels (hint: loop over `text.lower()` and test `ch in "aeiou"`).

QUICK CHECK: answer these in your notebook (answers at the back)

1. What do `7 // 2`, `7 % 2` and `7 / 2` evaluate to?
2. What is `'Python'[1:4]`?
3. Why does `word[0] = "J"` fail for a string?
4. What does `f"{2/3:.3f}"` produce?
5. What does `"ab" * 3` give?
6. What is the key difference between a list and a string?

CHAPTER 3

Control Flow: Decisions and Loops

Official docs for this chapter: https://docs.python.org/3/tutorial/controlflow.html

Programs become useful when they can make decisions and repeat actions. This chapter covers `if`, `for`, `while`, `range()`, `break`, `continue` and `match`.

3.1 if, elif and else

```
x = int(input("Please enter an integer: "))
if x < 0:
    print("Negative")
elif x == 0:
    print("Zero")
elif x == 1:
    print("Single")
else:
    print("More")
```

Comparison operators: `== != < <= > >=`. Combine tests with `and`, `or`, `not`. You can chain comparisons: `0 < x < 10`.

Truthiness: these values count as False: `0`, `0.0`, `""` (empty string), `[]`, `{}`, `set()`, `None` and `False`. Everything else is True.

```
name = ""
if not name:
    print("Name is empty")
```

3.2 for loops and range()

A `for` loop visits each item of a sequence, in order.

```
words = ['cat', 'window', 'defenestrate']
for w in words:
    print(w, len(w))
```

`range()` generates numbers:

```
>>> list(range(5))
[0, 1, 2, 3, 4]
>>> list(range(5, 10))
[5, 6, 7, 8, 9]
>>> list(range(0, 10, 3))      # start, stop, step
[0, 3, 6, 9]
>>> list(range(-10, -100, -30))
[-10, -40, -70]
```

When you need both the index and the item, use `enumerate()`; to walk two lists together use `zip()`:

```
for i, color in enumerate(['red', 'green', 'blue']):
    print(i, color)

for name, age in zip(['Ana', 'Raj'], [31, 27]):
    print(name, "is", age)
```

> **WATCH OUT** Do not add or remove items from a list while looping over it. Loop over a copy (`for x in items[:]`) or build a new list instead.

3.3 break, continue and loop else

`break` leaves the loop immediately; `continue` jumps to the next iteration. A loop `else` block runs only if the loop finished **without** hitting `break`. This is perfect for searching, as in this prime-number finder:

```
for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            print(n, "equals", x, "*", n // x)
            break
    else:
        # loop fell through without finding a factor
        print(n, "is a prime number")
```

3.4 while loops

```
total = 0
while True:
    s = input("Add a number (or q to stop): ")
    if s == "q":
        break
    total += float(s)
print("Total:", total)
```

3.5 pass and match

`pass` does nothing; it is a placeholder where Python needs a statement. `match` (Python 3.10+) compares a value against patterns:

```
def http_error(status):
    match status:
        case 400:
            return "Bad request"
        case 404:
            return "Not found"
        case 401 | 403:
            return "Not allowed"
        case _:
            return "Something is wrong with the internet"

point = (0, 5)
match point:
    case (0, 0):
        print("Origin")
    case (0, y):
        print(f"On the Y axis at {y}")
    case (x, 0):
        print(f"On the X axis at {x}")
    case (x, y):
        print(f"Somewhere else: {x}, {y}")
```

MINI PROJECT 4: FizzBuzz

The classic interview warm-up. Loop from 1 to 100.

Your program should:

- Print `Fizz` if the number is divisible by 3.
- Print `Buzz` if divisible by 5, and `FizzBuzz` if divisible by both.
- Otherwise print the number itself.

MINI PROJECT 5: Number Guessing Game

The computer picks a secret number from 1 to 100 and you try to guess it.

Your program should:

- Tell the player "Too low" or "Too high" after each guess.
- Count the attempts and show the total when the player wins.
- Allow only 7 attempts, then reveal the number.

Starter hint (try on your own first, peek only if stuck):

```
import random
secret = random.randint(1, 100)
tries = 0
while True:
    guess = int(input("Your guess: "))
    tries += 1
    # TODO: compare guess with secret and print a hint
    # TODO: break when the guess is correct, and show tries
```

MINI PROJECT 6: Prime Finder

Ask for a number N and print every prime number below N.

Your program should:

- Reuse the `for ... else` idea from section 3.3.
- Print how many primes you found.
- Challenge: only test divisors up to the square root of the number.

QUICK CHECK: answer these in your notebook (answers at the back)

1. What numbers does `range(2, 10, 3)` produce?
2. When does the `else` block of a loop run?
3. Name five values that are treated as False.
4. What is the difference between `break` and `continue`?
5. What does `case _:` mean in a `match` statement?

CHAPTER 4

Functions

Official docs for this chapter: https://docs.python.org/3/tutorial/controlflow.html#defining-functions

A function is a named, reusable block of code. Functions make programs shorter, easier to test and easier to read.

4.1 Defining and calling functions

```
def fib(n):
    """Return a list with the Fibonacci series up to n."""
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a + b
    return result

print(fib(100))
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

- The first string in a function is its **docstring**; `help(fib)` shows it.
- `return` sends a value back. A function without `return` gives back `None`.
- Variables created inside a function are **local**; they disappear when it ends.

```
counter = 0

def increment():
    global counter     # needed to *assign* to a module-level name
    counter += 1
```

4.2 Default values and keyword arguments

```
def describe_pet(name, animal="dog", age=None):
    text = f"{name} is a {animal}"
    if age is not None:
        text += f" aged {age}"
    return text

describe_pet("Rex")                       # Rex is a dog
describe_pet("Tom", "cat")                # Tom is a cat
describe_pet("Kiwi", age=2, animal="bird")  # keyword arguments, any order
```

WATCH OUT Default values are created **once**. A mutable default such as `L=[]` is shared between calls. Use `None` and create the list inside the function.

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

4.3 *args and **kwargs

```
def total(*numbers):          # numbers is a tuple
    return sum(numbers)

def show(**info):             # info is a dict
    for key, value in info.items():
        print(key, "=", value)

total(1, 2, 3)                # 6
show(name="Ana", city="Pune")

args = [3, 6]
list(range(*args))            # unpack a list into arguments: [3, 4, 5]
```

You can also force how arguments are passed: parameters before `/` are positional-only, parameters after `*` are keyword-only.

```
def f(pos1, pos2, /, pos_or_kwd, *, kwd1, kwd2):
    ...
```

4.4 Lambdas, annotations and style

A `lambda` is a small anonymous function, handy as a sort key:

```
pairs = [(1, 'one'), (2, 'two'), (3, 'three'), (4, 'four')]
pairs.sort(key=lambda pair: pair[1])
# [(4, 'four'), (1, 'one'), (3, 'three'), (2, 'two')]
```

Annotations document the expected types. Python does not enforce them, but editors and type checkers use them:

```
def greet(name: str, times: int = 1) -> str:
    return ("Hello " + name + "! ") * times
```

Coding style (PEP 8)

- Use 4 spaces per indent level, never tabs mixed with spaces.
- Keep lines to about 79 characters.
- `snake_case` for functions and variables, `CamelCase` for classes.
- Put spaces around operators and after commas: `a = b + c`, `f(x, y)`.
- Write a docstring for each function.

MINI PROJECT 7: Temperature Converter

Write two functions, `c_to_f(c)` and `f_to_c(f)`, and a small menu around them.

Your program should:

- Show a menu: 1) Celsius to Fahrenheit, 2) Fahrenheit to Celsius, 3) Quit.
- Repeat until the user chooses Quit.
- Print results with one decimal place.

Starter hint (try on your own first, peek only if stuck):

```
def c_to_f(c):
    return c * 9 / 5 + 32

def f_to_c(f):
    # TODO: write this function
    ...
```

MINI PROJECT 8: Password Strength Checker

Write `strength(password)` that returns a score from 0 to 5.

Your program should:

- +1 for length of 8 or more, +1 for a digit, +1 for an upper-case letter, +1 for a lower-case letter, +1 for a symbol.
- Print "Weak", "Medium" or "Strong" depending on the score.

Starter hint (try on your own first, peek only if stuck):

```
def strength(password):
    score = 0
    if len(password) >= 8:
        score += 1
    if any(ch.isdigit() for ch in password):
        score += 1
    # TODO: upper, lower, and symbols (hint: not ch.isalnum())
    return score
```

QUICK CHECK: answer these in your notebook (answers at the back)

1. What does a function return if it has no `return` statement?
2. Why is `def f(a, L=[])` a bad idea?
3. What do `*args` and `**kwargs` collect?
4. What does `/` mean in a parameter list?
5. Write a lambda that returns the square of x.

CHAPTER 5

Data Structures

Official docs for this chapter: https://docs.python.org/3/tutorial/datastructures.html

Python has four main built-in containers: **lists**, **tuples**, **sets** and **dictionaries**. Choosing the right one makes your code simpler and faster.

Type	Syntax	Ordered?	Changeable?	Use it for
list	`[1, 2, 3]`	yes	yes	a sequence of items
tuple	`(1, 2, 3)`	yes	no	fixed groups, e.g. a point
set	`{1, 2, 3}`	no	yes	unique items, fast membership
dict	`{"a": 1}`	insertion order	yes	look up values by key

5.1 More on lists

Method	Purpose
`append(x)`	add x to the end
`extend(iterable)`	add all items of another iterable
`insert(i, x)`	insert x before index i
`remove(x)`	remove the first item equal to x (ValueError if missing)
`pop([i])`	remove and return the item at i (default: the last)
`index(x)`, `count(x)`	position of first x; how many times x occurs
`sort()`, `reverse()`	sort or reverse in place
`copy()`, `clear()`	shallow copy; remove everything

```
>>> fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi', 'apple', 'banana']
>>> fruits.count('apple')
2
>>> fruits.index('banana')
3
>>> fruits.index('banana', 4)      # search starting at position 4
6
>>> fruits.reverse()
>>> fruits.append('grape')
>>> fruits.sort()
>>> fruits
['apple', 'apple', 'banana', 'banana', 'grape', 'kiwi', 'orange', 'pear']
>>> fruits.pop()
'pear'
```

NOTE Methods that change a list in place (`insert`, `remove`, `sort`...) return `None`. Do not write `x = x.sort()`. Use `sorted(x)` if you want a new sorted list.

Lists as stacks and queues

```
stack = [3, 4, 5]
stack.append(6)
stack.pop()              # 6  (last in, first out)

from collections import deque
queue = deque(["Eric", "John", "Michael"])
queue.append("Terry")
queue.popleft()          # 'Eric'  (first in, first out; fast)
```

List comprehensions

A concise way to build a list from another sequence:

```
>>> squares = [x ** 2 for x in range(10)]
>>> squares
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> evens = [x for x in range(20) if x % 2 == 0]
>>> [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
>>> matrix = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
>>> [[row[i] for row in matrix] for i in range(4)]   # transpose
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

The `del` statement removes items by index or slice: `del a[0]`, `del a[2:4]`, `del a[:]`.

5.2 Tuples

A tuple is like a list that cannot be changed. Tuples can be unpacked into variables.

```
t = 12345, 54321, 'hello!'      # tuple packing
x, y, z = t                     # sequence unpacking
single = ('hello',)             # a one-item tuple needs the trailing comma
empty = ()
a, b = 1, 2
a, b = b, a                     # swap values without a temp variable
```

5.3 Sets

A set stores unique items and supports fast membership tests and maths-style operations. Printing order may vary.

```
>>> basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
>>> basket                      # duplicates are gone
{'banana', 'orange', 'pear', 'apple'}
>>> 'orange' in basket
True
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a - b          # in a but not in b
{'r', 'd', 'b'}
>>> a | b          # in either
{'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
>>> a & b          # in both
{'a', 'c'}
>>> a ^ b          # in one but not both
{'r', 'd', 'b', 'm', 'z', 'l'}
```

> **WATCH OUT** `{}` creates an empty **dictionary**. For an empty set use `set()`.

5.4 Dictionaries

A dictionary maps **keys** to **values**. Keys must be immutable (strings, numbers, tuples). Dictionaries remember insertion order.

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel['jack']
4098
>>> del tel['sape']
>>> list(tel)
['jack', 'guido']
>>> 'guido' in tel
True
>>> tel.get('nobody', 0)         # safe lookup with a default
0
>>> {x: x ** 2 for x in (2, 4, 6)}
{2: 4, 4: 16, 6: 36}
>>> defaults = {'color': 'red', 'size': 1}
>>> defaults | {'size': 5}       # merge (Python 3.9+)
{'color': 'red', 'size': 5}
```

5.5 Looping techniques

```
knights = {'gallahad': 'the pure', 'robin': 'the brave'}
for k, v in knights.items():
    print(k, v)

for i, v in enumerate(['tic', 'tac', 'toe']):
    print(i, v)

questions = ['name', 'quest', 'favorite color']
answers = ['lancelot', 'the holy grail', 'blue']
for q, a in zip(questions, answers):
    print(f"What is your {q}? It is {a}.")

for i in reversed(range(1, 10, 2)):    # 9 7 5 3 1
    print(i)
for f in sorted({'pear', 'apple', 'orange'}):
    print(f)
```

5.6 Conditions and comparing sequences

- `in` and `not in` test membership; `is` tests whether two names point to the **same object** (use `==` to compare values).
- `and`/`or` short-circuit: `a or b` returns `a` if it is truthy, otherwise `b`.
- Sequences compare item by item (lexicographically): `(1, 2, 3) < (1, 2, 4)` is True, `[1, 2] < [1, 2, -1]` is True, `'ABC' < 'Pascal'` is True.

MINI PROJECT 9: Word Frequency Counter

Count how many times each word appears in a piece of text.

Your program should:

- Ask the user for a paragraph (or hard-code one).
- Convert to lower case and strip punctuation like `.,!?`.
- Store counts in a dictionary and print the top 5 words.

Starter hint (try on your own first, peek only if stuck):

```
text = input("Paste some text: ").lower()
counts = {}
for word in text.split():
    word = word.strip(".,!?;:\"'")
    counts[word] = counts.get(word, 0) + 1
top = sorted(counts.items(), key=lambda item: item[1], reverse=True)
for word, n in top[:5]:
    print(f"{word:<15}{n}")
```

MINI PROJECT 10: Phone Book

Build a menu-driven contact book that lives in a dictionary.

Your program should:

- Menu: add, look up, delete, list all (sorted by name), quit.
- Do not crash if the name does not exist (use `get()` or `in`).
- Bonus: store several phone numbers per person using a list as the value.

MINI PROJECT 11: Friends in Common

Given two lists of friends, use sets to answer questions.

Your program should:

- Who are the friends both people have in common?
- Who is only in the first list? Who is in either list (without duplicates)?
- Print each answer in sorted order.

Starter hint (try on your own first, peek only if stuck):

```
alice = {"Ben", "Cara", "Dev", "Eli"}
bob = {"Cara", "Eli", "Fay"}
print("Common:", sorted(alice & bob))
# TODO: alice only, everyone combined
```

QUICK CHECK: answer these in your notebook (answers at the back)

1. What is the difference between `list.sort()` and `sorted(list)`?
2. How do you create an empty set?
3. What does `[x * 2 for x in range(4)]` produce?
4. Why use `d.get("key", 0)` rather than `d["key"]`?
5. Which data type would you use to remove duplicates from a list?
6. What does `a, b = b, a` do?

CHAPTER 6

Modules and Packages

Official docs for this chapter: https://docs.python.org/3/tutorial/modules.html

As programs grow you split them into files. A **module** is simply a `.py` file; its definitions can be imported into other files. A **package** is a folder of modules.

6.1 Writing and importing a module

Create `fibo.py`:

```
# fibo.py - Fibonacci numbers module

def fib(n):
    """Write the Fibonacci series up to n."""
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a + b
    print()

def fib2(n):
    """Return the Fibonacci series up to n."""
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a + b
    return result
```

Now use it from another file or from the interpreter (in the same folder):

```
>>> import fibo
>>> fibo.fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
>>> fibo.__name__
'fibo'
>>> from fibo import fib, fib2        # import specific names
>>> import fibo as fb                  # give the module a short alias
>>> from fibo import fib as fibonacci
```

6.2 Running a module as a script

Add this at the bottom of a module so that it works both as an import and as a program:

```
if __name__ == "__main__":
    import sys
    fib(int(sys.argv[1]))
```

`python fibo.py 50` prints `0 1 1 2 3 5 8 13 21 34`. The block does **not** run when the module is imported, because then `__name__` is `"fibo"`, not `"__main__"`.

6.3 Where does Python look for modules?

When you `import spam`, Python looks for a built-in module first, then in the folders listed in `sys.path`: the folder of the running script, the `PYTHONPATH` environment variable and the installation defaults. Do not name your own file after a standard module (for example `random.py`), or you will shadow the real one.

`dir(module)` lists the names a module defines, and `help(module)` shows its documentation.

6.4 Packages

```
sound/                  top-level package
    __init__.py         marks the folder as a package
    formats/
        __init__.py
        wavread.py
    effects/
        __init__.py
        echo.py
```

```
import sound.effects.echo               # use full name: sound.effects.echo.func()
from sound.effects import echo          # use short name: echo.func()
from . import echo                      # relative import (inside the package)
from ..formats import wavread           # one level up
```

TIP Avoid `from module import *`; it makes it unclear where names come from.

MINI PROJECT 12: Build Your Own Utility Module

Create two files in one folder: `mytools.py` and `main.py`.

Your program should:

- `mytools.py` has three functions: `is_even(n)`, `average(numbers)` and `shout(text)` (upper-case plus "!").
- `main.py` imports them (try both import styles) and shows each in action.
- Add an `if __name__ == "__main__":` block in `mytools.py` with a few self-tests.

Starter hint (try on your own first, peek only if stuck):

```
# main.py
import mytools
from mytools import shout

print(mytools.is_even(10))
print(mytools.average([4, 8, 15, 16, 23, 42]))
print(shout("modules are neat"))
```

QUICK CHECK: answer these in your notebook (answers at the back)

1. What is the difference between a module and a package?
2. What does `if __name__ == "__main__":` achieve?
3. How do you give an imported module a shorter name?
4. What function lists the names defined in a module?

CHAPTER 7

Input, Output and Files

Official docs for this chapter: https://docs.python.org/3/tutorial/inputoutput.html

7.1 Formatting output

f-strings (formatted string literals) put expressions inside `{}`. After a colon you can add a format specification:

```
>>> import math
>>> f'Pi is about {math.pi:.3f}'
'Pi is about 3.142'
>>> name, phone = 'Sjoerd', 4127
>>> f'{name:10} ==> {phone:10d}'
'Sjoerd     ==>       4127'
>>> f'{name:>10}|{name:<10}|{name:^10}'      # right, left, centre
'    Sjoerd|Sjoerd    |  Sjoerd  '
>>> f'{1234567:,}'                           # thousands separator
'1,234,567'
>>> f'{0.256:.1%}'                           # percent
'25.6%'
>>> bugs, count = 'roaches', 13
>>> f'{bugs=} {count=}'                      # debugging aid
"bugs='roaches' count=13"
```

Older styles still appear in existing code: `"{} and {}".format("a", "b")` and `"%d items" % 3`. Also handy: `str(x)` for humans, `repr(x)` for programmers, and `"7".zfill(3)` which gives `"007"`.

7.2 Reading and writing files

Always open files with a `with` block: the file is closed automatically, even if an error happens. Specify `encoding="utf-8"` for text.

Mode	Meaning
`"r"`	read (default); error if the file does not exist
`"w"`	write; creates the file or **erases** an existing one
`"a"`	append to the end of the file
`"r+"`	read and write
add `"b"`	binary mode, e.g. `"rb"` for images

```
# writing
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("First line\n")
    f.write("Second line\n")

# reading everything
with open("notes.txt", encoding="utf-8") as f:
    content = f.read()

# reading line by line (memory friendly)
with open("notes.txt", encoding="utf-8") as f:
    for line in f:
```

```
        print(line, end="")      # line already ends with \n

# appending
with open("notes.txt", "a", encoding="utf-8") as f:
    f.write("Third line\n")
```

> **TIP** `pathlib` offers an object-oriented alternative: `Path("notes.txt").read_text()` and `Path("notes.txt").write_text("hi")`.

7.3 Saving structured data with JSON

JSON is a text format for lists, dictionaries, strings and numbers. It is the easiest way to save program data.

```
import json

data = {"name": "Ana", "scores": [90, 85], "active": True}

with open("data.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)

with open("data.json", encoding="utf-8") as f:
    loaded = json.load(f)
print(loaded["scores"])          # [90, 85]
```

> **NOTE** JSON turns tuples into lists and dictionary keys into strings. `json.dumps(x)` returns a string; `json.loads(s)` parses one.

MINI PROJECT 13: Daily Journal

Create a journal program that stores entries in `journal.txt`.

Your program should:

- Menu: write a new entry, read all entries, quit.
- Every entry is saved with a timestamp using `datetime.now().strftime("%Y-%m-%d %H:%M")`.
- Reading must not crash if the file does not exist yet.

Starter hint (try on your own first, peek only if stuck):

```
from datetime import datetime

def add_entry(text):
    stamp = datetime.now().strftime("%Y-%m-%d %H:%M")
    with open("journal.txt", "a", encoding="utf-8") as f:
        f.write(f"[{stamp}] {text}\n")
```

MINI PROJECT 14: Shopping List with Memory

A shopping list that remembers items between runs using JSON.

Your program should:

- Load the list from `shopping.json` at start (start empty if the file is missing).
- Let the user add an item, remove an item, show the list and quit.
- Save the list back to the file when quitting.

QUICK CHECK: answer these in your notebook (answers at the back)

1. Why should you open files with `with open(...)`?
2. Which mode would you use to add lines to an existing file without erasing it?
3. What does `f"{3.14159:.2f}"` print?
4. What happens to tuples when you save them as JSON?

CHAPTER 8

Errors and Exceptions

Official docs for this chapter: https://docs.python.org/3/tutorial/errors.html

There are two kinds of errors. **Syntax errors** are found before the program runs (a missing colon, for example). **Exceptions** happen while the program runs, for example dividing by zero. Exceptions can be handled so your program does not crash.

```
>>> 10 * (1/0)
ZeroDivisionError: division by zero
>>> 4 + spam*3
NameError: name 'spam' is not defined
>>> '2' + 2
TypeError: can only concatenate str (not "int") to str
>>> int("abc")
ValueError: invalid literal for int() with base 10: 'abc'
```

8.1 try, except, else, finally

```
while True:
    try:
        x = int(input("Please enter a number: "))
        break
    except ValueError:
        print("Oops! That was not a valid number. Try again...")
```

```
try:
    f = open("data.txt", encoding="utf-8")
except OSError as err:                  # catch several: except (A, B):
    print("Cannot open file:", err)
else:                                   # runs only if no exception happened
    print("File has", len(f.readlines()), "lines")
    f.close()
finally:                                # ALWAYS runs: clean-up code
    print("Done")
```

TIP Catch the most specific exception you can. A bare `except:` hides real bugs.

8.2 Raising your own exceptions

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

try:
    set_age(-5)
except ValueError as e:
    print("Problem:", e)
    # use a bare `raise` here to re-raise the same exception
```

Use `raise NewError(...) from original` to **chain** exceptions, so the traceback shows what caused what.

8.3 User-defined exceptions

```
class InsufficientFunds(Exception):
    """Raised when a withdrawal is bigger than the balance."""

class Account:
    def __init__(self, balance=0):
        self.balance = balance
    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFunds(f"Balance is only {self.balance}")
        self.balance -= amount
```

8.4 More tools

- `with` statements give predefined clean-up (files, locks, connections).
- `ExceptionGroup` and `except*` handle several unrelated errors at once (Python 3.11+).
- `e.add_note("extra context")` attaches information to an exception before re-raising it.

MINI PROJECT 15: Crash-Proof Calculator

A calculator loop that never crashes, however silly the input.

Your program should:

- Ask for two numbers and an operator (`+ - * /`) in a loop; type `q` to quit.
- Handle `ValueError` (not a number) and `ZeroDivisionError` with friendly messages.
- Reject unknown operators by raising and catching your own exception.

Starter hint (try on your own first, peek only if stuck):

```
while True:
    try:
        a = float(input("First number: "))
        op = input("Operator (+ - * /): ")
        b = float(input("Second number: "))
        # TODO: compute and print the result
    except ValueError:
        print("Please type a number.")
    # TODO: ZeroDivisionError, quit option
```

QUICK CHECK: answer these in your notebook (answers at the back)

1. What is the difference between a syntax error and an exception?
2. When does the `else` clause of `try` run?
3. When does `finally` run?
4. How do you create your own exception type?

CHAPTER 9

Classes and Objects

Official docs for this chapter: https://docs.python.org/3/tutorial/classes.html

A **class** is a blueprint; an **object** (instance) is something built from it. Classes bundle data (attributes) and behaviour (methods) together. This is called object-oriented programming.

9.1 Your first class

```
class Dog:
    kind = "canine"            # class variable: shared by all dogs

    def __init__(self, name):  # runs when you create a Dog
        self.name = name       # instance variable: unique to each dog
        self.tricks = []

    def add_trick(self, trick):
        self.tricks.append(trick)

    def __str__(self):         # what print() shows
        return f"{self.name} knows {len(self.tricks)} tricks"

d = Dog("Fido")
e = Dog("Buddy")
d.add_trick("roll over")
print(d)            # Fido knows 1 tricks
print(e.tricks)     # []   - each dog has its own list
print(d.kind)       # canine
```

- `self` is the object the method is working on. It is always the first parameter.
- `__init__` is the **constructor**; names with double underscores are "special methods".
- `__str__` gives the friendly text and `__repr__` the developer text of an object.

WATCH OUT A mutable class variable such as `tricks = []` written at class level would be shared by **every** instance. Create per-object lists inside `__init__`.

9.2 Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."

class Cat(Animal):             # Cat inherits from Animal
    def speak(self):           # override the parent method
        return "Meow"

class Puppy(Animal):
    def __init__(self, name, age):
        super().__init__(name)  # call the parent's __init__
        self.age = age
    def speak(self):
        return "Woof"
```

```
pets = [Cat("Tom"), Puppy("Rex", 1), Animal("Thing")]
for pet in pets:
    print(pet.name, pet.speak())      # polymorphism in action

isinstance(pets[0], Animal)     # True
issubclass(Cat, Animal)         # True
```

9.3 Private names and dataclasses

Python has no strict private attributes. By convention `_balance` means "internal, please do not touch". A name with two leading underscores, `__secret`, is mangled to `_ClassName__secret` to avoid accidental clashes in subclasses.

For classes that mainly store data, `dataclass` writes the boring methods for you:

```
from dataclasses import dataclass

@dataclass
class Employee:
    name: str
    dept: str
    salary: int = 1000

john = Employee("john", "computer lab")
print(john)        # Employee(name='john', dept='computer lab', salary=1000)
```

9.4 Scopes: where is a name visible?

Python looks a name up in this order, known as **LEGB**: **L**ocal (inside the function), **E**nclosing functions, **G**lobal (module level), **B**uilt-in. Assignments create local names unless you declare `global` or `nonlocal`. Also remember that assignment never copies: `b = a` makes both names point to the **same** object.

9.5 Iterators and generators

Every `for` loop uses an **iterator**: an object with `__iter__` and `__next__`.

```
class Reverse:
    """Iterator for looping over a sequence backwards."""
    def __init__(self, data):
        self.data = data
        self.index = len(data)
    def __iter__(self):
        return self
    def __next__(self):
        if self.index == 0:
            raise StopIteration
        self.index -= 1
        return self.data[self.index]

for char in Reverse("spam"):
    print(char, end=" ")        # m a p s
```

A **generator** does the same with far less code. A function that uses `yield` produces values one at a time and remembers where it stopped:

```
def reverse(data):
    for index in range(len(data) - 1, -1, -1):
```

```
        yield data[index]

for char in reverse("golf"):
    print(char, end=" ")        # f l o g
```

Generator expressions look like list comprehensions with parentheses and do not build the whole list in memory:

```
>>> sum(i * i for i in range(10))
285
>>> xvec, yvec = [10, 20, 30], [7, 5, 3]
>>> sum(x * y for x, y in zip(xvec, yvec))      # dot product
260
```

MINI PROJECT 16: Bank Account

Write a `BankAccount` class.

Your program should:

- Attributes: `owner` and `balance` (start at 0).
- Methods: `deposit(amount)`, `withdraw(amount)` and `__str__`.
- Raise `ValueError` for negative deposits and a custom `InsufficientFunds` exception when withdrawing too much.
- Create two accounts and test every case with `try/except`.

Starter hint (try on your own first, peek only if stuck):

```
class InsufficientFunds(Exception):
    pass

class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        # TODO: validate amount, then add it
        ...
```

MINI PROJECT 17: Countdown and Fibonacci Generators

Practise `yield`.

Your program should:

- Write a generator `countdown(n)` that yields n, n-1, ... 1.
- Write `fib_gen()` that yields Fibonacci numbers forever.
- Use `itertools.islice(fib_gen(), 15)` to print the first 15 numbers.

QUICK CHECK: answer these in your notebook (answers at the back)

1. What is `self`?
2. What is the difference between a class variable and an instance variable?
3. How does a subclass call its parent's `__init__`?
4. What does the keyword `yield` do?
5. Which special method controls what `print(obj)` shows?

CHAPTER 10

A Tour of the Standard Library

Official docs for this chapter: https://docs.python.org/3/tutorial/stdlib.html

Python is famous for being "batteries included": a huge standard library ships with it. Here are the modules you will use most. Import them and explore with `help()` and `dir()`.

10.1 Files, folders and the system

```
import os, shutil, glob, sys
from pathlib import Path

os.getcwd()                         # current folder
os.listdir(".")                     # names in a folder
shutil.copyfile("data.db", "archive.db")
shutil.move("/build/a.txt", "/build/b.txt")
glob.glob("*.py")                   # ['main.py', 'utils.py', ...]
sys.argv                            # command-line arguments as a list

p = Path("notes") / "today.txt"     # build paths with /
p.exists(), p.suffix, p.name        # inspect them
```

10.2 Command-line programs with argparse

```
import argparse

parser = argparse.ArgumentParser(prog="top",
                                 description="Show top lines from each file")
parser.add_argument("filenames", nargs="+")
parser.add_argument("-l", "--lines", type=int, default=10)
args = parser.parse_args()
print(args.filenames, args.lines)
# python top.py -l 5 alpha.txt beta.txt
```

10.3 Text patterns, maths and randomness

```
import re, math, random, statistics

re.findall(r"\bf[a-z]*", "which foot or hand fell fastest")
# ['foot', 'fell', 'fastest']
re.sub(r"(\b[a-z]+) \1", r"\1", "cat in the the hat")
# 'cat in the hat'

math.sqrt(16), math.log(1024, 2), math.pi
random.choice(["apple", "pear", "banana"])
random.sample(range(100), 5)        # 5 unique numbers
random.randrange(6)                 # 0..5
statistics.mean([2.5, 3.25, 5.75])  # 3.5
statistics.median([2.5, 3.25, 5.75]) # 3.25
```

10.4 Dates and times

```
from datetime import date, datetime, timedelta

today = date.today()
print(today.strftime("%d-%m-%Y, %A"))        # e.g. 19-10-2026, Monday
birthday = date(2000, 5, 17)
age_days = (today - birthday).days
next_week = today + timedelta(days=7)
now = datetime.now()
```

10.5 Collections and itertools

```
from collections import Counter, deque, defaultdict
from itertools import permutations, islice

Counter("mississippi").most_common(2)   # [('i', 4), ('s', 4)]

groups = defaultdict(list)              # missing keys start as []
for word in ["apple", "avocado", "banana"]:
    groups[word[0]].append(word)
# {'a': ['apple', 'avocado'], 'b': ['banana']}

list(permutations("abc", 2))            # [('a','b'), ('a','c'), ('b','a'), ...]
```

10.6 Quality control: testing

Tests prove your code works and keep working. `unittest` ships with Python:

```
import unittest

def average(values):
    return sum(values) / len(values)

class TestAverage(unittest.TestCase):
    def test_average(self):
        self.assertEqual(average([20, 30, 70]), 40.0)
        self.assertEqual(round(average([1, 5, 7]), 1), 4.3)
        with self.assertRaises(ZeroDivisionError):
            average([])

if __name__ == "__main__":
    unittest.main()
```

10.7 More batteries

Module	What it is for
`logging`	professional status/debug/error messages instead of `print`
`json`, `csv`	read and write JSON and spreadsheet-style files
`urllib.request`	download web pages and data (needs an internet connection)
`decimal`	exact decimal arithmetic, ideal for money
`threading`	run tasks concurrently, such as downloads in the background
`timeit`	measure how fast a snippet of code runs
`zlib`, `zipfile`	compress data and work with zip archives
`textwrap`, `string.Template`	wrap text and simple text templates
`secrets`	cryptographically safe random values, e.g. passwords
`hashlib`	hashes such as SHA-256 to fingerprint files or data

```
import logging
logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
logging.info("Program started")
logging.warning("Config file %s not found", "server.conf")
```

MINI PROJECT 18: File Organiser

Tidy a messy folder by sorting files into sub-folders by extension (`.jpg` goes into `jpg/`, `.pdf` into `pdf/`).

Your program should:

- Ask for a folder path and use `pathlib`.
- Create sub-folders as needed and move files with `shutil.move`.
- Print a summary like "Moved 14 files".
- **Safety:** test on a copy of a folder first!

Starter hint (try on your own first, peek only if stuck):

```
from pathlib import Path
import shutil

folder = Path(input("Folder to organise: "))
for item in folder.iterdir():
    if item.is_file():
        ext = item.suffix.lower().lstrip(".") or "no_extension"
        target = folder / ext
        target.mkdir(exist_ok=True)
        shutil.move(str(item), str(target / item.name))
```

MINI PROJECT 19: Dice Roller Command-Line Tool

Create `dice.py` that you run as `python dice.py --count 3 --sides 6`.

Your program should:

- Use `argparse` with `--count` (default 1) and `--sides` (default 6).
- Roll with `random.randint` and print each roll plus the total.

Starter hint (try on your own first, peek only if stuck):

```
import argparse, random

parser = argparse.ArgumentParser(description="Roll some dice")
parser.add_argument("--count", type=int, default=1)
parser.add_argument("--sides", type=int, default=6)
args = parser.parse_args()
# TODO: roll the dice and print the results
```

QUICK CHECK: answer these in your notebook (answers at the back)

1. Which module would you use to build file paths safely on any operating system?
2. What does `Counter("hello").most_common(1)` return?
3. What is the purpose of `argparse`?
4. Why prefer `secrets` over `random` for passwords?
5. Which module lets you write automated tests that ship with Python?

CHAPTER 11

Virtual Environments, pip and Floating-Point

Official docs for this chapter: https://docs.python.org/3/tutorial/venv.html

11.1 Virtual environments

Different projects need different packages and versions. A **virtual environment** is an isolated folder with its own Python packages, so projects never break each other. Always create one per project.

```
# 1. create it (the folder name .venv is a common choice)
python -m venv .venv

# 2. activate it
source .venv/bin/activate        # Linux / macOS
.venv\Scripts\activate           # Windows

# 3. install packages from PyPI
python -m pip install requests
python -m pip install --upgrade requests
python -m pip show requests

# 4. save and restore your project's packages
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt

# 5. leave the environment
deactivate
```

11.2 Floating-point surprises

Computers store decimals in binary, so some decimal fractions cannot be exact. This is not a Python bug; every language behaves the same.

```
>>> 0.1 + 0.1 + 0.1 == 0.3
False
>>> 0.1 + 0.2
0.30000000000000004
>>> format(0.1, ".17f")
'0.10000000000000001'
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)      # compare floats like this
True
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2")   # exact: use for money
Decimal('0.3')
```

TIP Never compare floats with `==`. Use `math.isclose`, or round to a fixed number of digits, or switch to `decimal` / `fractions`.

11.3 What to learn next

- **Read the docs**: the Library Reference (standard modules) and the Language Reference.
- **Python HOWTOs**: in-depth guides, such as Logging, Sorting, Regular Expressions.
- **PEP 8** (style guide) and **PEP 20**: type `import this` in the REPL.
- **PyPI** (pypi.org) has hundreds of thousands of third-party packages.
- **Build projects**: the 20 projects in Part II of this book are your next step.

MINI PROJECT 20: Set Up a Professional Workspace

Practise the workflow every real project uses.

Your program should:

- Make a folder `my_project`, create a virtual environment inside it and activate it.
- Install one third-party package with pip (for example `requests`) and import it in a script to confirm it works.
- Save the dependencies with `pip freeze > requirements.txt`.
- Delete the `.venv` folder, recreate it and restore everything from `requirements.txt`.

QUICK CHECK: answer these in your notebook (answers at the back)

1. Why use a virtual environment?
2. Which command saves the installed packages to a file?
3. Why is `0.1 + 0.2 == 0.3` False?
4. What is the best way to compare two floats?

PART II

Twenty Projects to Build

You now know the language. The only way to become fluent is to build things. These 20 projects grow from beginner to advanced. Each one lists requirements, the skills it practises, stretch goals and a small hint. There are no solutions to copy: figuring it out yourself is the learning.

How to attack any project

- **Understand**: say in one sentence what the program must do.
- **Plan**: list the features and the functions or classes you will need.
- **Build the smallest working version first**, then add one feature at a time.
- **Test after every change**: run it, try weird input, read each error message slowly.
- **Stuck?** Print variables, search the docs, take a break, then explain the problem out loud ("rubber duck debugging").
- **Finish with the stretch goals** and keep your code in a Git repository.

#	Project	Level
1	Rock, Paper, Scissors Tournament	Beginner
2	Smart Calculator	Beginner
3	Password Generator and Checker	Beginner
4	Hangman	Beginner
5	Unit Converter	Beginner
6	Contact Book	Beginner / Intermediate
7	Quiz Engine	Intermediate
8	Expense Tracker	Intermediate
9	Cipher Tool: Caesar and Vigenere	Intermediate
10	Text Statistics Command-Line Tool	Intermediate
11	Text Adventure Game	Intermediate
12	Tic-Tac-Toe vs the Computer	Intermediate
13	Bank Account System	Intermediate
14	Library Management System	Intermediate
15	Student Grade Manager	Intermediate
16	Log File Analyser	Intermediate / Advanced
17	File Organiser and Duplicate Finder	Intermediate / Advanced
18	Sudoku Solver	Advanced
19	Markdown to HTML Converter	Advanced
20	Capstone: Inventory Manager with Tests	Advanced

Project 1: Rock, Paper, Scissors Tournament

Level: Beginner

Play against the computer in a best-of-N match.

Requirements

- Ask how many rounds to play (best of 3, 5 or 7).
- The computer picks with `random.choice`; the player types rock, paper or scissors.
- Show the result of every round and the running score; announce the match winner.
- Reject invalid input and ask again.

Skills practised: loops, `random`, dictionaries (what beats what), functions

Stretch goals:

- Add "lizard" and "Spock".
- Save win/loss statistics to a file.

Hint (a tiny piece to get you started):

```
beats = {"rock": "scissors", "paper": "rock", "scissors": "paper"}
# if beats[player] == computer: player wins the round
```

Project 2: Smart Calculator

Level: Beginner

A calculator that understands whole expressions such as `12 + 3 * 4`.

Requirements

- Read one line of input and split it into number, operator, number.
- Support + - * / // % ** and show a clear message on errors.
- Keep a history list; typing `history` prints previous calculations.
- Type `quit` to exit.

Skills practised: functions, dictionaries of operations, exceptions, lists

Stretch goals:

- Allow the previous answer to be used as `ans`.
- Support parentheses and operator precedence (research the `ast` module, but never use `eval` on untrusted input).

Hint (a tiny piece to get you started):

```
import operator
ops = {"+": operator.add, "-": operator.sub, "*": operator.mul, "/": operator.truediv}
# result = ops[symbol](a, b)
```

Project 3: Password Generator and Checker

Level: Beginner

Generate strong random passwords and rate existing ones.

Requirements

- Ask for the length and whether to include digits and symbols.
- Build the password with the `secrets` module and `string.ascii_letters`, `string.digits`, `string.punctuation`.
- Guarantee at least one character from every chosen group.
- Add a checker that scores any password you type from 0 to 5.

Skills practised: `secrets`, `string`, functions, loops

Stretch goals:

- Generate 5 options at once.
- Reject passwords found in a small list of common ones.

Hint (a tiny piece to get you started):

```
import secrets, string
alphabet = string.ascii_letters + string.digits
password = "".join(secrets.choice(alphabet) for _ in range(12))
```

Project 4: Hangman

Level: Beginner

The classic word-guessing game in the terminal.

Requirements

- Pick a random word from a list (or from a text file).
- Show the word as `_ a _ _ e _` and the letters already guessed.
- Allow 6 wrong guesses; draw simple ASCII art for the gallows.
- Ignore repeated guesses and invalid input; offer a replay.

Skills practised: strings, sets, loops, lists, files

Stretch goals:

- Add difficulty levels by word length.
- Load 1000 words from a file.

Hint (a tiny piece to get you started):

```
word = "python"
guessed = set()
shown = " ".join(ch if ch in guessed else "_" for ch in word)
```

Project 5: Unit Converter

Level: Beginner

Convert between lengths, weights and temperatures.

Requirements

- Show a menu of categories, then ask for the "from" unit, "to" unit and value.
- Store conversion factors in dictionaries instead of long `if` chains.
- Handle temperature separately (it is not a simple multiplication).
- Validate input and show friendly errors.

Skills practised: dictionaries, functions, input validation

Stretch goals:

- Add currencies with rates typed in by the user.
- Let the user type "10 km to miles" as one sentence.

Hint (a tiny piece to get you started):

```
to_metres = {"m": 1, "km": 1000, "cm": 0.01, "mile": 1609.344, "ft": 0.3048}
# metres = value * to_metres[from_unit]; result = metres / to_metres[to_unit]
```

Project 6: Contact Book

Level: Beginner / Intermediate

Store names, phone numbers and emails and save them to disk.

Requirements

- Add, search (partial matches too), edit, delete and list contacts.
- Save to `contacts.json` after every change; load it on start-up.
- Prevent duplicate names and validate the email contains `@`.
- Present a clean, aligned table of results using f-string formatting.

Skills practised: dictionaries, JSON, functions, string methods

Stretch goals:

- Export to CSV.
- Add birthdays and a "who has a birthday this month?" feature.

Hint (a tiny piece to get you started):

```
contacts = {"Ana": {"phone": "99999", "email": "ana@example.com"}}
matches = {n: d for n, d in contacts.items() if "an" in n.lower()}
```

Project 7: Quiz Engine

Level: Intermediate

A multiple-choice quiz that loads questions from a JSON file.

Requirements

- Create `questions.json` with question, four options and the correct answer.
- Shuffle questions and options each run.
- Score the player, show the correct answers at the end, and store a high-score table.
- Let the user choose a category and number of questions.

Skills practised: JSON, lists of dictionaries, `random.shuffle`, file I/O, functions

Stretch goals:

- Add a timer per question.
- Add a "create new question" mode.

Hint (a tiny piece to get you started):

```
# questions.json
[{"q": "Which keyword defines a function?",
  "options": ["func", "def", "lambda", "fn"], "answer": "def"}]
```

Project 8: Expense Tracker

Level: Intermediate

Record spending and see where the money goes.

Requirements

- Add an expense: date, amount, category, note. Store in `expenses.csv`.
- Show totals per category and per month.
- List the 5 biggest expenses and filter by date range.
- Use the `csv` module and `datetime`.

Skills practised: `csv`, `datetime`, dictionaries, sorting, file I/O

Stretch goals:

- Set a monthly budget and warn when it is exceeded.
- Print a text bar chart: `Food ########## 4200`.

Hint (a tiny piece to get you started):

```
import csv
from collections import defaultdict
totals = defaultdict(float)
with open("expenses.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        totals[row["category"]] += float(row["amount"])
```

Project 9: Cipher Tool: Caesar and Vigenere

Level: Intermediate

Encrypt and decrypt secret messages.

Requirements

- Implement a Caesar cipher with a chosen shift; keep case and leave punctuation alone.
- Implement the Vigenere cipher with a keyword.
- Add "brute force" mode that prints all 25 Caesar shifts of an unknown message.
- Write functions `encrypt(text, key)` and `decrypt(text, key)`.

Skills practised: strings, `ord()` / `chr()`, modulo arithmetic, functions

Stretch goals:

- Detect the most likely shift using letter frequency.
- Encrypt whole files.

Hint (a tiny piece to get you started):

```
def shift_char(ch, shift):
    if ch.isalpha():
        base = ord("A") if ch.isupper() else ord("a")
        return chr((ord(ch) - base + shift) % 26 + base)
    return ch
```

Project 10: Text Statistics Command-Line Tool

Level: Intermediate

Analyse any text file from the command line: `python stats.py book.txt`.

Requirements

- Count lines, words, characters and sentences.
- Show the 10 most common words, ignoring common "stop words" like the, and, of.
- Report average word length and the longest word.
- Use `argparse` with options like `--top 20` and `--min-length 4`.

Skills practised: `argparse`, `collections.Counter`, `re`, file I/O

Stretch goals:

- Compare two files.
- Write the report to a file with `--output`.

Hint (a tiny piece to get you started):

```
import re
from collections import Counter
words = re.findall(r"[a-z']+", text.lower())
print(Counter(words).most_common(10))
```

Project 11: Text Adventure Game

Level: Intermediate

Build an interactive story with rooms, items and choices.

Requirements

- Model the world as a dictionary: each room has a description and exits.
- Commands: `go north`, `take lamp`, `inventory`, `look`, `quit`.
- Some doors need an item (a key) to open; add at least 6 rooms and a winning condition.
- Use functions and a game-state dictionary rather than global variables.

Skills practised: dictionaries, loops, string parsing, functions, `match` (optional)

Stretch goals:

- Save and load games with JSON.
- Add a simple enemy with hit points.

Hint (a tiny piece to get you started):

```
rooms = {
    "hall": {"text": "A dusty hall.", "exits": {"north": "library"}, "items": ["key"]},
    "library": {"text": "Books everywhere.", "exits": {"south": "hall"}, "items": []},
}
state = {"room": "hall", "inventory": []}
```

Project 12: Tic-Tac-Toe vs the Computer

Level: Intermediate

Two-player and one-player versions of the classic 3x3 game.

Requirements

- Draw the board in the terminal after every move.
- Validate moves (empty squares only) and detect wins and draws.
- Computer opponent: first random, then "win if you can, block if you must".
- Keep the board in a list of 9 items and use a tuple of winning lines.

Skills practised: lists, functions, loops, tuples, `random`

Stretch goals:

- Make the computer unbeatable using the minimax algorithm.
- Keep a scoreboard across games.

Hint (a tiny piece to get you started):

```
WIN_LINES = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
def winner(board):
    for a, b, c in WIN_LINES:
        if board[a] != " " and board[a] == board[b] == board[c]:
            return board[a]
```

Project 13: Bank Account System

Level: Intermediate

An object-oriented banking simulation.

Requirements

- Base class `Account` with deposit, withdraw and a transaction history.
- Subclasses `SavingsAccount` (adds interest) and `CheckingAccount` (allows an overdraft limit).
- Raise custom exceptions such as `InsufficientFunds`.
- A `Bank` class stores many accounts, finds them by number and can transfer money.

Skills practised: classes, inheritance, `super()`, custom exceptions, `datetime`

Stretch goals:

- Persist accounts in JSON.
- Write unit tests for every rule.

Hint (a tiny piece to get you started):

```
class Account:
    def __init__(self, number, owner, balance=0):
        self.number, self.owner, self.balance = number, owner, balance
        self.history = []
```

Project 14: Library Management System

Level: Intermediate

Track books, members and loans.

Requirements

- Classes: `Book`, `Member`, `Library`.
- Features: add books, register members, borrow, return, search by title or author.
- A member may borrow at most 3 books; a book can only be borrowed once at a time.
- Save everything to JSON and reload it on start.

Skills practised: classes, composition, JSON, list/dict handling, exceptions

Stretch goals:

- Add due dates and fines for late returns using `datetime`.
- Report the most borrowed books.

Hint (a tiny piece to get you started):

```
from dataclasses import dataclass

@dataclass
class Book:
    isbn: str
    title: str
    author: str
    available: bool = True
```

Project 15: Student Grade Manager

Level: Intermediate

Keep marks for many students and subjects and print report cards.

Requirements

- Store students as a dictionary of subject-to-marks.
- Compute average, highest, lowest and a letter grade per student.
- Rank the class and print a neatly formatted table.
- Use the `statistics` module for mean, median and standard deviation.

Skills practised: dictionaries of dictionaries, `statistics`, sorting, f-string tables

Stretch goals:

- Import marks from a CSV file.
- Export report cards as text files, one per student.

Hint (a tiny piece to get you started):

```
import statistics
marks = {"Ana": {"math": 88, "physics": 92}, "Raj": {"math": 75, "physics": 81}}
for name, subjects in marks.items():
    print(f"{name:<10}{statistics.mean(subjects.values()):6.1f}")
```

Project 16: Log File Analyser

Level: Intermediate / Advanced

Find insights in a server log with regular expressions.

Requirements

- Create (or download) a sample web-server log with lines like `192.168.1.5 - - [19/Oct/2026:10:15:32] "GET /home HTTP/1.1" 200 512`.
- Parse each line with a regex into IP, time, method, path, status and size.
- Report: top 5 IP addresses, top pages, number of 404 and 500 errors, requests per hour.
- Accept the log file path as a command-line argument.

Skills practised: `re` with named groups, `Counter`, `defaultdict`, `argparse`

Stretch goals:

- Flag IPs with suspiciously many failed requests.
- Generate a summary report file.

Hint (a tiny piece to get you started):

```
import re
pattern = re.compile(r'(?P<ip>\S+) \S+ \S+ \[(?P<time>[^\]]+)\] '
                     r'"(?P<method>\w+) (?P<path>\S+) [^"]*" '
                     r'(?P<status>\d{3}) (?P<size>\d+)')
m = pattern.match(line)
if m:
    data = m.groupdict()
```

Project 17: File Organiser and Duplicate Finder

Level: Intermediate / Advanced

Take Mini Project 18 further and find duplicate files.

Requirements

- Scan a folder tree recursively with `Path.rglob("*")`.
- Identify duplicates by computing a SHA-256 hash of each file (read in chunks).
- Print groups of duplicates with their sizes and the space you could save.
- Offer a "dry run" mode and only delete after explicit confirmation.

Skills practised: `pathlib`, `hashlib`, dictionaries, `argparse`, safe file handling

Stretch goals:

- Move duplicates to a "trash" folder instead of deleting.
- Sort files into folders by modified date.

Hint (a tiny piece to get you started):

```
import hashlib
def file_hash(path, chunk=65536):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        while block := f.read(chunk):
            h.update(block)
    return h.hexdigest()
```

Project 18: Sudoku Solver

Level: Advanced

Solve any valid 9x9 Sudoku puzzle using backtracking.

Requirements

- Represent the grid as a list of 9 lists of 9 integers (0 means empty).
- Write `is_valid(grid, row, col, num)` that checks row, column and 3x3 box.
- Write a recursive `solve(grid)` that tries numbers and backtracks on failure.
- Print the grid nicely, with separators between boxes.

Skills practised: recursion, nested lists, algorithms, functions

Stretch goals:

- Load puzzles from text files.
- Count how many steps were needed; try the "most constrained cell first" idea.

Hint (a tiny piece to get you started):

```
def find_empty(grid):
    for r in range(9):
        for c in range(9):
            if grid[r][c] == 0:
                return r, c
    return None
```

Project 19: Markdown to HTML Converter

Level: Advanced

Convert a small subset of Markdown into HTML.

Requirements

- Headings (`# ... ###`), paragraphs, **bold**, *italic*, and `inline code`.
- Bullet lists (`- item`) turned into `<ul><li>` blocks.
- Links `[text](url)` and horizontal rules.
- Read `input.md`, write `output.html`, and escape special characters like `<` and `&`.

Skills practised: `re.sub` with groups, string processing, state handling, file I/O

Stretch goals:

- Support numbered lists and code blocks.
- Add a command-line interface with `argparse`.

Hint (a tiny piece to get you started):

```
import re
text = re.sub(r"\*\*(.+?)\*\*", r"<strong>\1</strong>", text)
text = re.sub(r"\[(.+?)\]\((.+?)\)", r'<a href="\2">\1</a>', text)
```

Project 20: Capstone: Inventory Manager with Tests

Level: Advanced

A complete, well-structured command-line application. Treat it like a real project.

Requirements

- Project layout: `inventory/` package with `models.py`, `storage.py`, `cli.py`; a `tests/` folder; a `requirements.txt`; a virtual environment.
- Classes `Product` and `Inventory`: add, remove, update stock, search, low-stock report.
- Persist to JSON; handle missing or corrupted files gracefully with exceptions.
- CLI with `argparse` sub-commands: `add`, `list`, `sell`, `report`.
- Log actions with the `logging` module; write at least 10 `unittest` tests.

Skills practised: everything: modules, packages, classes, exceptions, files, `argparse`, `logging`, `unittest`, virtual environments

Stretch goals:

- Export reports to CSV.
- Add type annotations and run a type checker such as mypy.

Hint (a tiny piece to get you started):

```
# python -m inventory add "USB cable" --qty 40 --price 4.99
# python -m inventory sell "USB cable" --qty 3
# python -m inventory report --low-stock 5
# python -m unittest discover tests
```

APPENDIX A

Python Cheat Sheet

Topic	Syntax
Variables	`x = 5` `name = "Ada"` `a, b = 1, 2`
Types	`int` `float` `str` `bool` `list` `tuple` `set` `dict` `None`
Conversion	`int("5")` `float("2.5")` `str(10)` `list("abc")`
Input / output	`name = input("Name? ")` `print(a, b, sep=", ", end="")`
Condition	`if x > 0:` / `elif x == 0:` / `else:`
for loop	`for i in range(5):` `for k, v in d.items():`
while loop	`while cond:` ... `break` / `continue`
Function	`def add(a, b=0):` ... `return a + b`
Lambda	`square = lambda x: x * x`
List ops	`l.append(x)` `l.pop()` `l.sort()` `l[1:3]` `len(l)`
Comprehension	`[x * 2 for x in nums if x > 0]` `{k: v for k, v in pairs}`
Dict ops	`d["k"] = v` `d.get("k", 0)` `del d["k"]` `"k" in d`
Set ops	`a | b` `a & b` `a - b` `a ^ b`
String ops	`s.split()` `s.strip()` `s.replace(a, b)` `"-".join(l)` `f"{x:.2f}"`
Files	`with open(p, "r", encoding="utf-8") as f:` `f.read()` `f.write(s)`
JSON	`json.dump(obj, f)` `obj = json.load(f)`
Exceptions	`try:` / `except ValueError as e:` / `else:` / `finally:` `raise ValueError("msg")`
Class	`class A(Base):` `def __init__(self, x):` `self.x = x`
Imports	`import os` `from math import sqrt` `import numpy as np`
Main guard	`if __name__ == "__main__":`
Venv + pip	`python -m venv .venv` `python -m pip install pkg`
Help	`help(obj)` `dir(obj)` `type(x)`

APPENDIX B

Answer Key for the Quick Checks

Chapter 1

1. `python --version` (or `python3 --version` on some systems).
2. Interactive mode runs each line as you type it at the `>>>` prompt; a script is a saved `.py` file that Python runs from top to bottom.
3. The hash sign `#`.
4. A string (`str`). Convert it with `int()` or `float()` if you need a number.
5. `exit()` or Ctrl-D (Ctrl-Z then Enter on Windows).

Chapter 2

1. `3`, `1` and `3.5`.
2. `'yth'` (index 1 up to, but not including, 4).
3. Strings are immutable; you cannot change characters in place.
4. `'0.667'` (three digits after the decimal point).
5. `'ababab'`.
6. Lists are mutable (changeable); strings are immutable.

Chapter 3

1. `2, 5, 8`.
2. When the loop ends normally, i.e. it was **not** stopped by `break`.
3. `0`, `0.0`, `""`, `[]`, `{}`, `set()`, `None` and `False` (any five).
4. `break` exits the loop; `continue` skips to the next iteration.
5. It is the wildcard pattern: it matches anything, so it works as the default case.

Chapter 4

1. `None`.
2. The default list is created once and shared between all calls, so items pile up.
3. `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict.
4. Parameters before it are positional-only.
5. `lambda x: x ** 2`

Chapter 5

1. `sort()` changes the list in place and returns `None`; `sorted()` returns a new list.
2. `set()`. (`{}` makes an empty dict.)
3. `[0, 2, 4, 6]`.
4. `get` returns the default instead of raising `KeyError` when the key is missing.
5. A set, e.g. `list(set(items))` (order is not kept).
6. Swaps the values of a and b.

Chapter 6

1. A module is a single `.py` file; a package is a folder of modules (with an `__init__.py`).
2. The block runs only when the file is executed directly, not when it is imported.
3. `import module as alias`.
4. `dir(module)`.

Chapter 7

1. The file is closed automatically, even if an exception occurs.
2. `"a"`.
3. `3.14`.
4. They come back as lists.

Chapter 8

1. A syntax error stops the program from starting; an exception occurs while it runs and can be handled.
2. Only when the `try` block raised no exception.
3. Always, whether or not an exception happened.
4. Subclass `Exception`: `class MyError(Exception): pass`.

Chapter 9

1. The instance the method is called on; always the first parameter.
2. A class variable is shared by all instances; an instance variable (set via `self.x`) belongs to one object.
3. `super().__init__(...)`.
4. It turns a function into a generator that returns values one at a time and pauses between them.
5. `__str__`.

Chapter 10

1. `pathlib` (or `os.path`).
2. `[('l', 2)]`.
3. To read command-line options and arguments and generate help text automatically.
4. `secrets` is designed to be cryptographically secure and unpredictable.
5. `unittest`.

Chapter 11

1. It isolates each project's packages so versions do not conflict.
2. `python -m pip freeze > requirements.txt`.
3. Binary floating point cannot represent these decimals exactly, so a tiny error appears.
4. `math.isclose(a, b)`.

APPENDIX C

Map of the Official Documentation

Where to find things at https://docs.python.org/3/

Section	Use it when you want to...
Tutorial (/tutorial/)	learn the language step by step (the base of this book)
Built-ins reference (/builtins/)	look up functions like `len`, `sorted`, `enumerate`, `zip`
Library reference (/library/)	look up any standard module: `os`, `re`, `json`, `datetime`...
Language reference (/reference/)	understand exact syntax and rules
Setup and usage (/using/)	install and configure Python on your system
HOWTOs (/howto/)	read deep guides: logging, sorting, regex, argparse, descriptors
Installing modules (/installing/)	use pip and PyPI
FAQs (/faq/)	get answers to common questions and gotchas
Glossary (/glossary.html)	decode terms such as iterable, decorator, duck typing
Module index (/py-modindex.html)	browse every standard module alphabetically
PEP index (https://peps.python.org/)	read design documents, including PEP 8 and PEP 20

Congratulations on reaching the end. Keep building, keep reading other people's code, and keep the habit of reading the documentation. Happy coding!